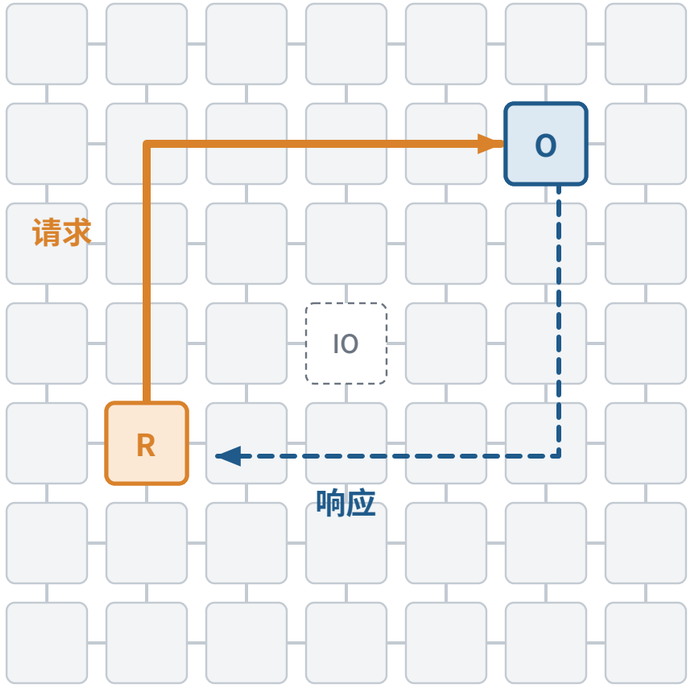
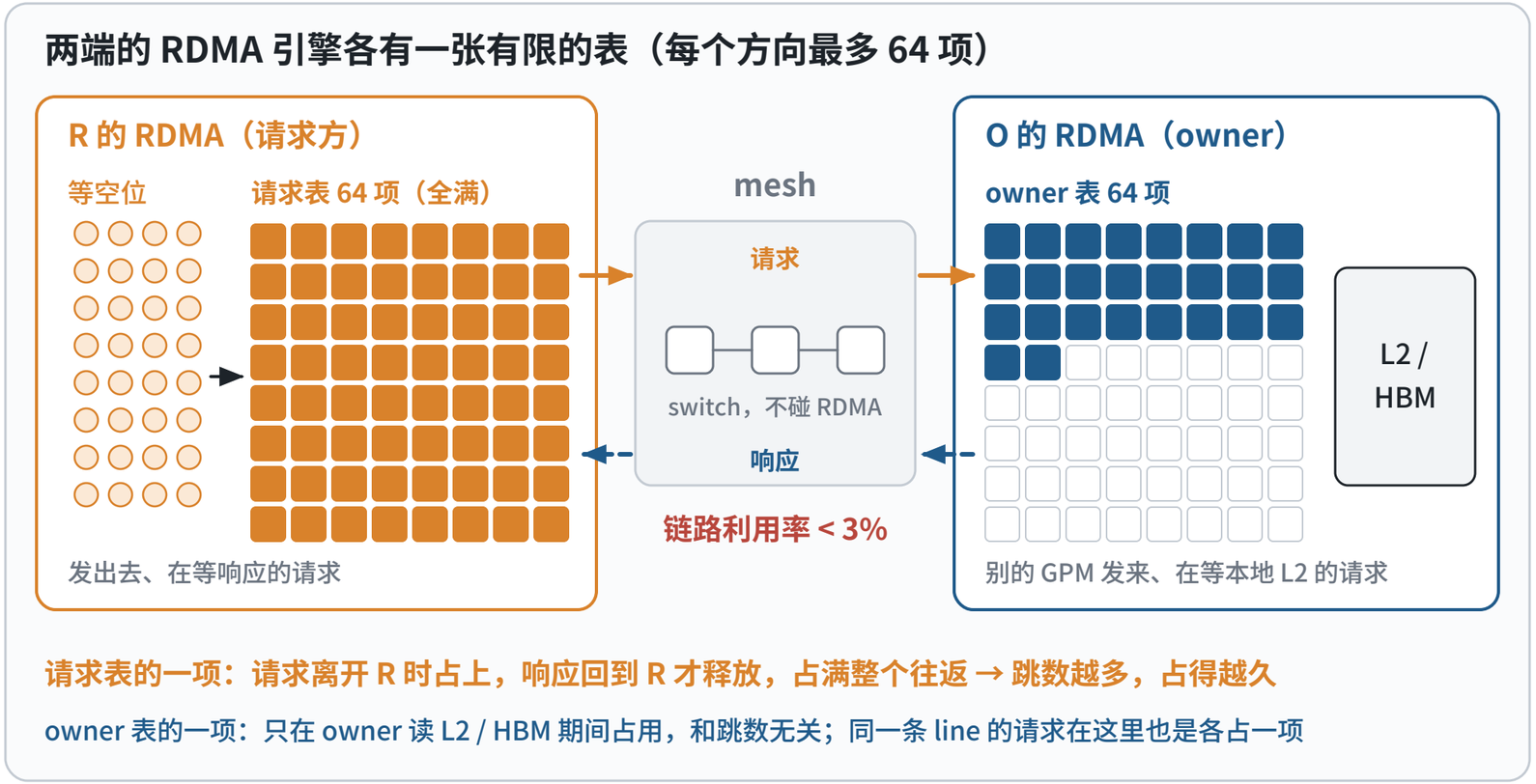
图 1 基线的瓶颈：请求方的 RDMA 引擎只能同时挂 64 个远程请求

RDMA 引擎是每个 GPM 连到 mesh 的网络接口，两端各有一张有限的表；中间每一跳只过 switch，不碰 RDMA



这一次 7 跳：只有 R、O 两端经过 RDMA，
中间每跳只过 switch（约 72 ns / 跳）



带宽被表项卡住

跳数越多，往返越长，每项占得越久：

远程带宽 = 64 × 64 B ÷ 往返时间

1 跳 26.4 B/周期，6 跳只剩 8.0

时间花在排队上

按闭环模型，排队占远程延迟 80–86%

（T17 的三个 workload）

排队就发生在请求表门口

链路是闲的

链路利用率 < 3%

受限的是两端 RDMA 的表，

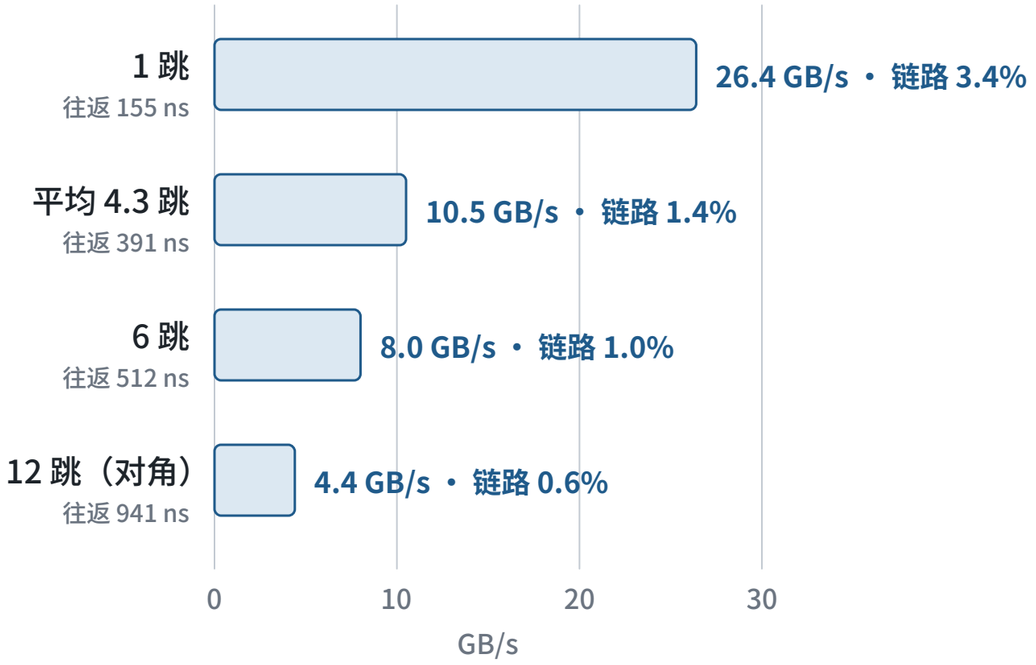
不是链路和 switch

图 2 为什么 wafer 上更严重：64 项在远处撑不起带宽，远的请求还会占满表项

一项从请求发出占到响应回来；跳数越多，每项占得越久

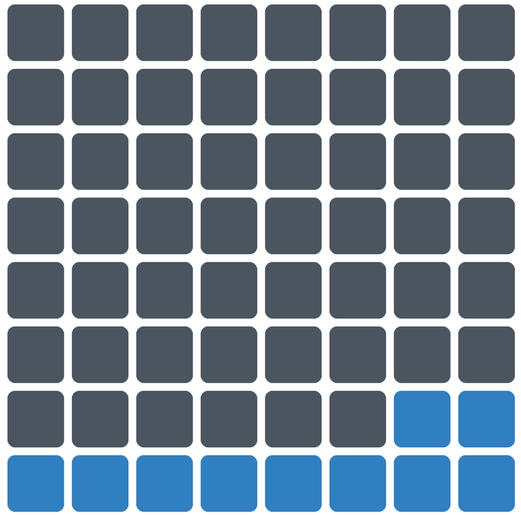
(a) 64 项能撑起的远程带宽

带宽 = 64 × 64 B ÷ 往返时间，往返 ≈ 84 + 71.4 ns × 跳数



(b) 共享 64 项时，远的请求占满表项

例子：一半请求去 1 跳外（往返 155 ns），一半去 10 跳外（往返 798 ns）



远的：54 项

近的：只剩 10 项

表项按"请求速率 × 往返时间"分，
远的每项占得久，就占了 84%；
近的 155 ns 就能完成，
却要排在后面等

2×2 的 MCM 最多 2 跳，往返最多差 1.5 倍；
wafer 上差 6 倍（155–941 ns）

64 这个数不改（用户决定，当作硬件约束）。M1 的三部分都在 64 项以内想办法：

(a) 请求表按远近分区：近的请求总有位置，不被远的占满

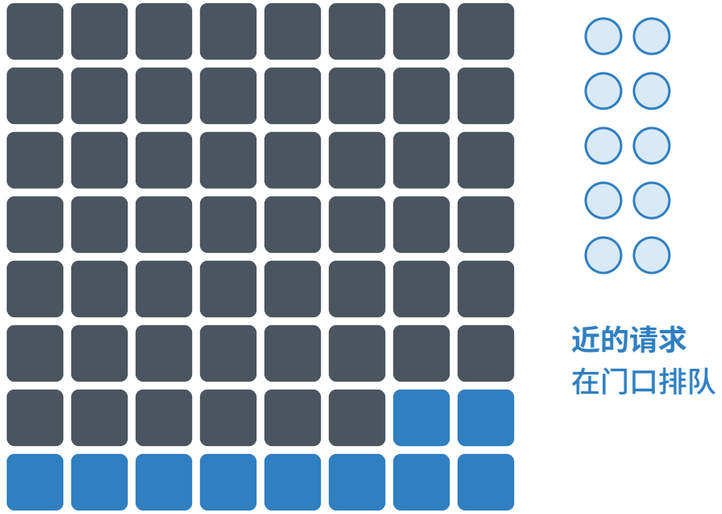
(b) owner 端合并：L2 读完之前到达的同 line 请求并成一项，只占 owner 一项、只读一次 L2

(c) 网内多播：合并后 owner 只发一份响应，在路由树的 switch 里复制给每个请求方

图 3 M1 (a)：请求表按远近分区（表还是 64 项）

近的请求总有位置；空着的分区可以互借，不浪费表项

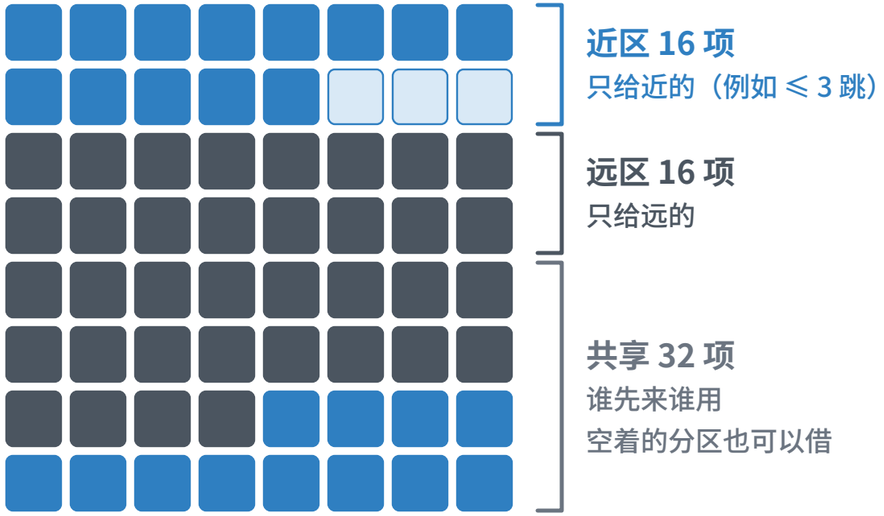
现在：共享 64 项



近的请求
在门口排队

远的占了 54 项；近的只有 10 项

M1 (a)：分区 + 可借用（比例是示意，实验来定）



近的请求不再排队；远的最多占远区 + 共享区

怎么做

1. 目的 GPM 由地址决定，跳数是已知的，不用预测
2. 近的先用近区，满了用共享区；远的同理
3. 按 Little 定律，近的往返短，同样的项每秒完成更多请求
4. 不需要 cuckoo filter：这是同一张表的分配策略

要说清楚的

每个 WG 远近数据都要时，总时间仍由最远的决定；

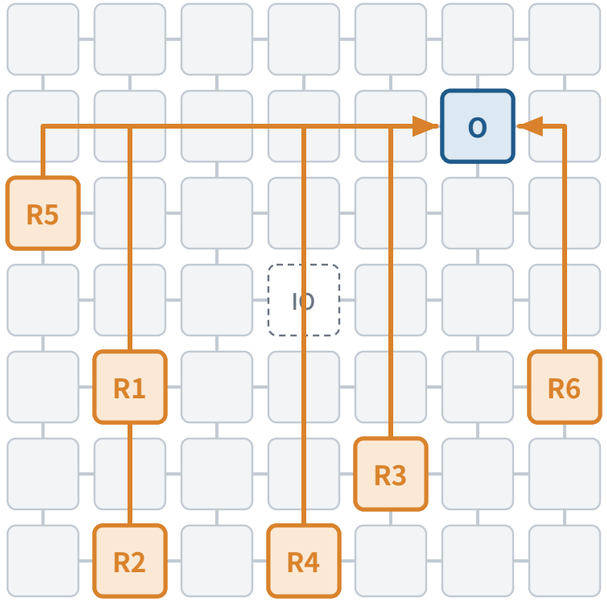
收益主要来自近的数据先到、先算，重叠更多

分多少、按几跳分，要用占用统计和实验来定

T18 现有的"按距离分 credit"是给远的更多，方向相反

图 4 M1 (b)：owner 端合并，L2 读完之前到达的同 line 请求并成一项

owner 表还是 64 项；合并窗口 = 这条 line 的 L2 读还没完成（T18 的实现）



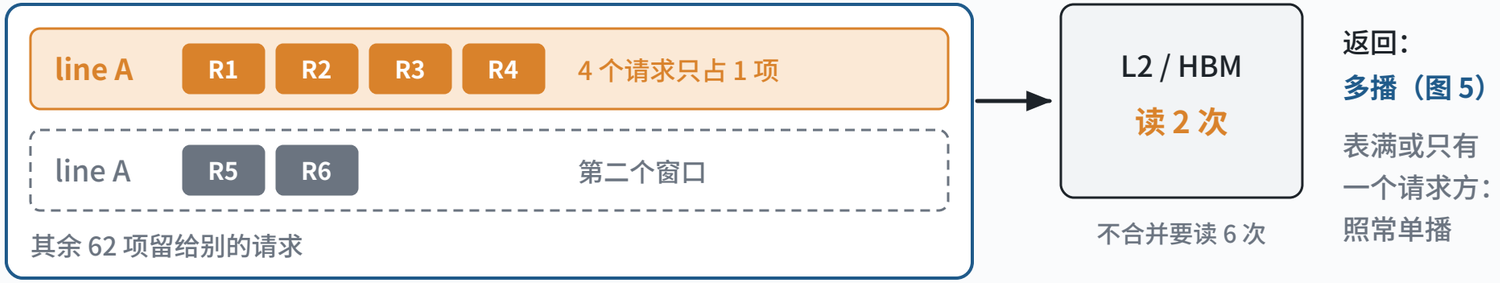
6 个 GPM 先后读 O 上的 line A

一条 line 最多 47 个读者（MCM 最多 3 个）

O 的 RDMA（owner 端）：line A 的请求先后到达



owner 表（64 项）



现有数据（64 项）

T18 @64（含 (b)(c)）：FW 2.13×，R-GAT 1.36×，PA 1.01×

只开 owner 端：FW 88.3 → 87.8 µs，几乎没用

owner 端同时在途的同 line 请求：PA 0.1%，FW 19%

→ 64 项时，同 line 请求很少能同时到 owner

为什么对 wafer 有用

incast：owner 封顶约 113 B/周期（≥ 8 个读者）

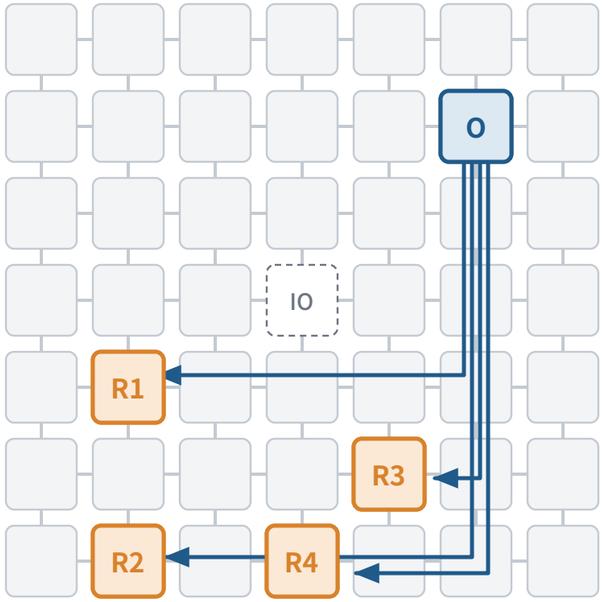
一条 line 最多 47 个读者，MCM 最多 3 个

合并后 owner 表和 L2 都只为这条 line 忙一次

图 5 M1 (c)：网内多播，合并后的一份响应在路由树上分叉

owner 只发一份响应；switch 按 YX 路由复制给每个请求方；没合并的照常单播（T18 的实现）

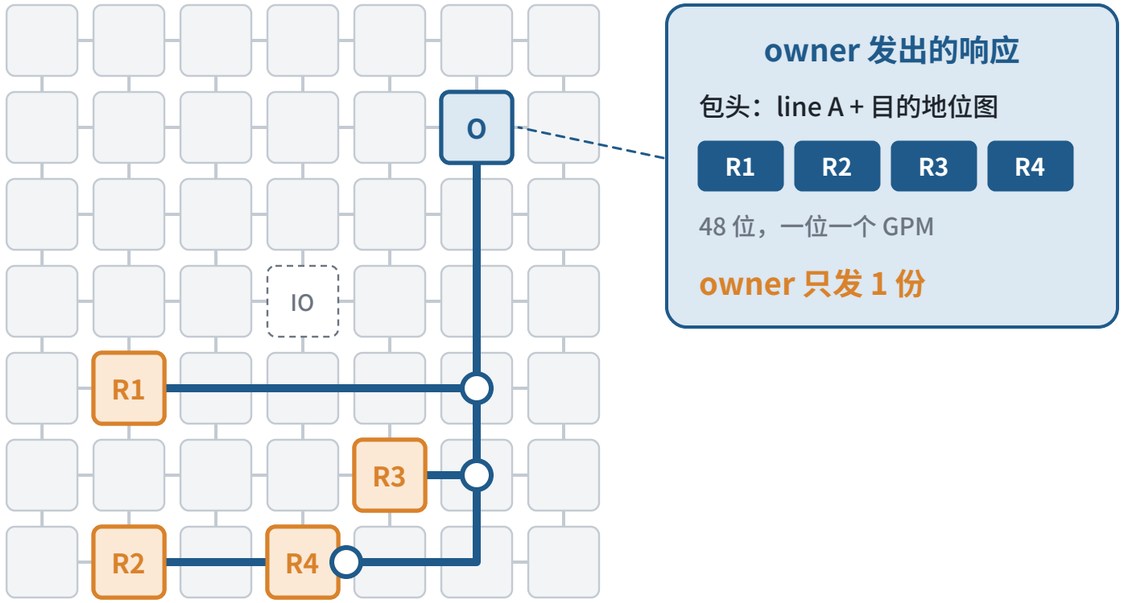
(a) 没有多播：4 份响应各走各的



owner 的 RDMA 发 4 份，共 28 段链路

同一段链路上最多叠 4 份同样的数据

(b) 多播：一份响应沿路由树分叉



一份响应在 switch 里复制（圆点处分叉），共 14 段链路

到了请求方 GPM 交给它的 RDMA，认领、唤醒等待者

switch 怎么做

1. 包头带目的地位图（来自 (b) 的 owner 合并表）
2. 按 YX 路由把位图按输出端口拆开，各复制一份
3. 只有一个目的地，或表满没合并的：普通单播

为什么对 wafer 有用

一条 line 最多 47 个读者

读者之间最近只隔 1.5–2.6 跳

共享 line 上省 25–52% 链路

要说清楚的

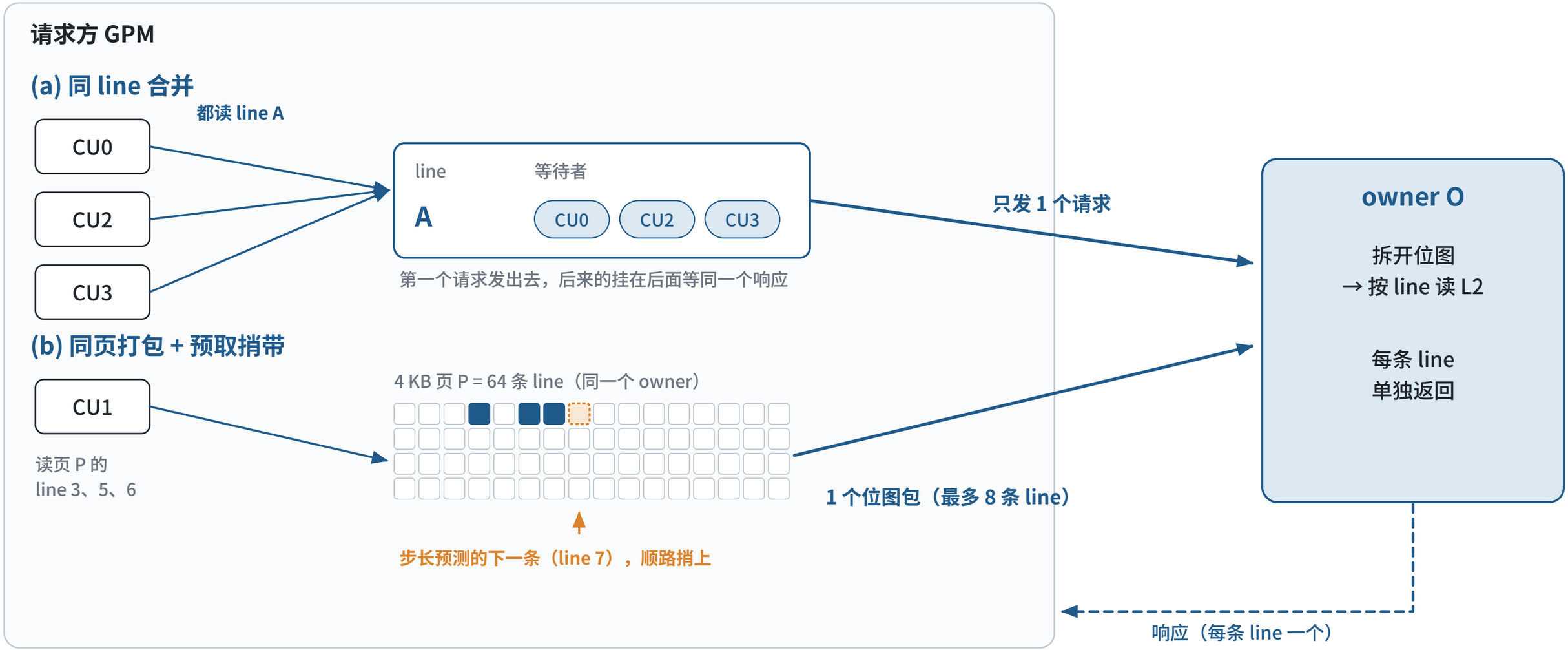
链路本来就闲（利用率 < 3%）

收益主要是 owner 少发几份响应

只有 (b) 合并了的才能多播

图 6 M2（HPCA 版 Remote Aggregation，已有）：在请求方合并、打包

同一个 GPM 发出的远程读，在进入 mesh 之前先去重，再把同一页的 line 打成一个包

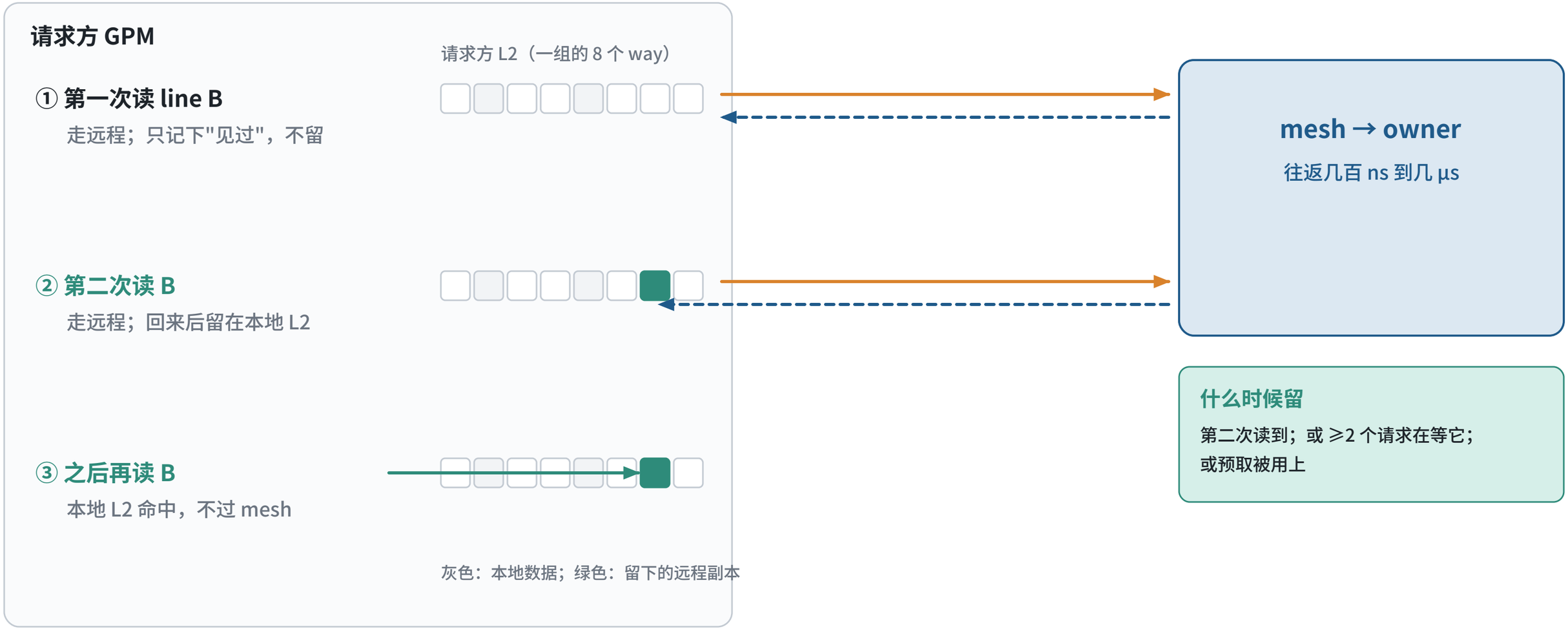


实现：每个包最多 8 条 line，最多 64 个正在收集的页；预测器只在同一 owner、同一页里预测，不多发包。

在 Complete 里，同 line 合并交给 M1 的跟踪表（M2 自己的 filter 去重关掉），M2 保留打包和预取。

图 7 M3（HPCA 版 Remote Reuse，已有，非重点）：看到复用才留在本地 L2

同一个 GPM 再次读到同一条远程 line 时，用本地 L2 里的副本，不再穿过晶圆

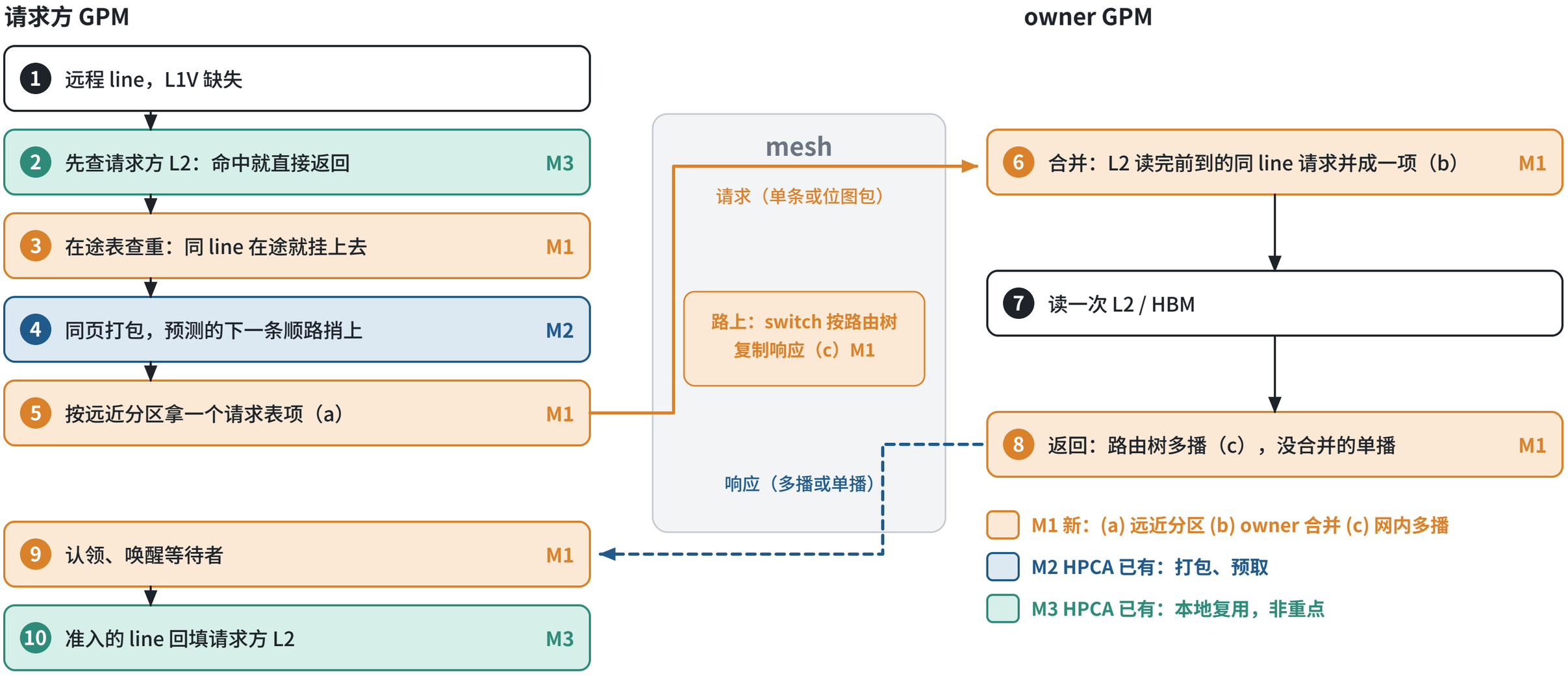


只占无效的 way 或替换别的远程副本，不挤掉本地数据；写照常发给 owner，本地副本一起更新。

还要补：kernel 边界的失效（论文写了，实现还没有）；模拟器的 L2 命中偏快（129 ns），M3 的收益可能偏高。

图 8 Complete = M1 + M2 + M3：一次远程读经过的每一步（RDMA 两端的表都是 64 项）

颜色表示这一步由哪个机制负责

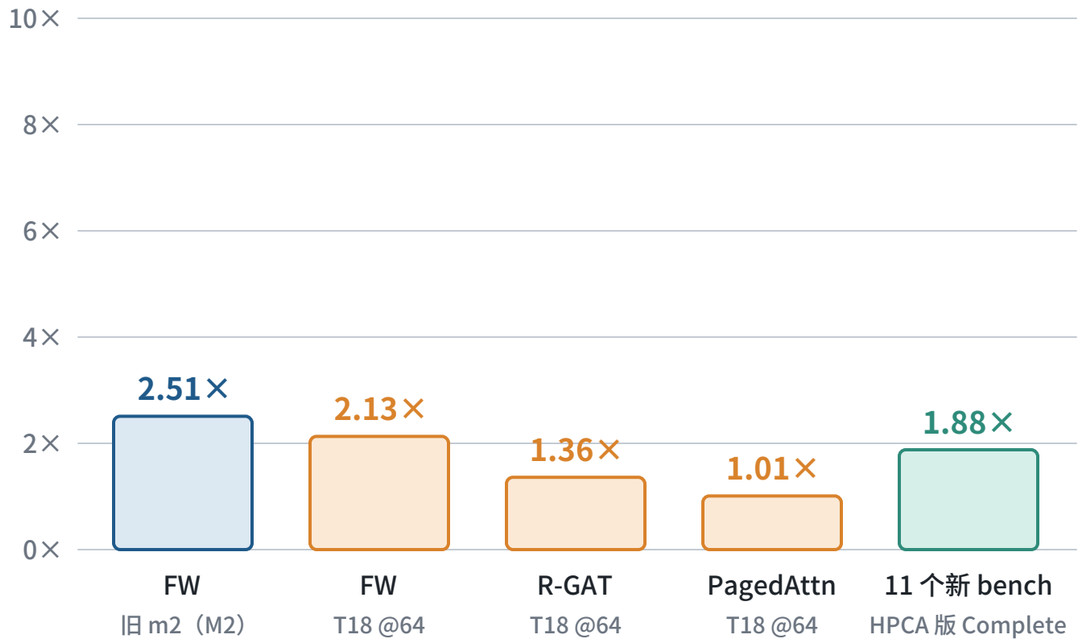


第 3 步用的是请求方 64 项在途表（cuckoo 索引）；第 2、10 步来自 M3；Local Pairing 不进论文

图 9 现有数据（表 64 项）和还缺的数据

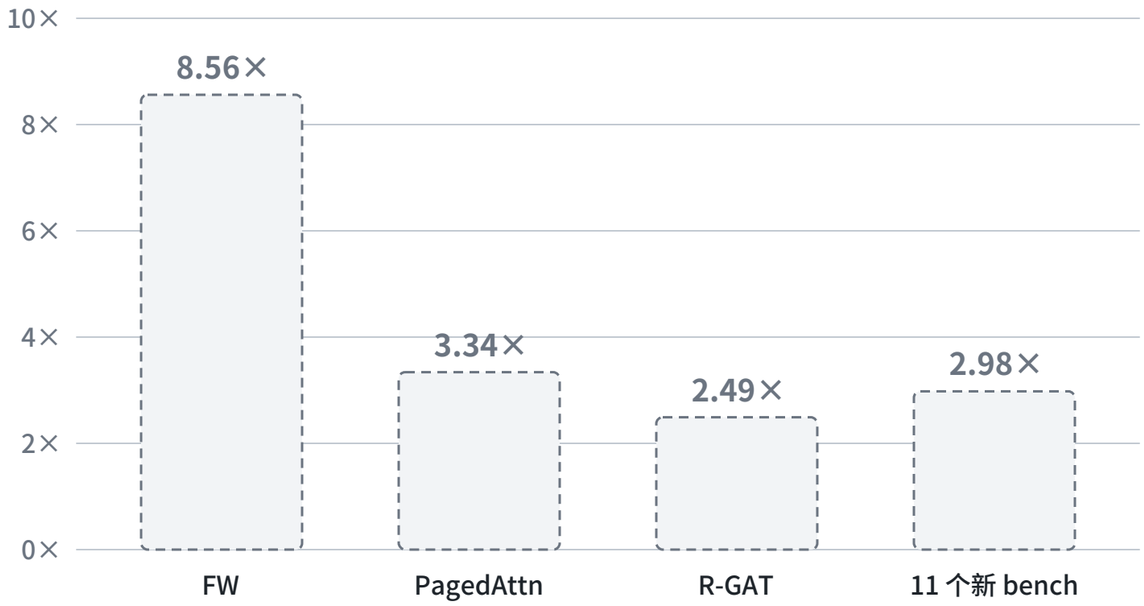
放大表项的结果只当"64 项这个限制值多少"的上限参考，不作为机制

(a) 表 64 项下的现有结果



T18 @64 = 请求方合并 + (b) owner 合并 + (c) 网内多播；还没有 (a) 远近分区

(b) 上限参考：表放大到 1024 项（不作为机制）



差距说明 64 项的限制很贵：M1 要在不改 64 的前提下拿回其中一部分

还缺（先测，只加统计、表保持 64 项）：

① 按距离分组的请求表占用：多少时间表满、被远的占着，同时近的在排队 → (a)

② owner 端实际合并了多少请求（合并窗口内到达的同 line 请求）、owner 表满的时间 → (b)

③ 多播响应的目的地个数、owner 少发的响应数、退回单播的比例 → (c)